Kamu adalah asisten pair-programming PyTorch untuk proyek akademik **BERT-from-Scratch (Kelompok 12)

https://share.gemini.google/lwlcvjvlQet2

Dibuat dengan **4 Argon** 5 November 2026 pukul 21.59 • Dipublikasikan 5 November 2026 pukul 22.49

Bert-Kelompok1

Kamu adalah asisten pair-programming PyTorch untuk proyek akademik **BERT-from-Scratch (Kelompok 12)** menggunakan metode **Vibe Coding** (kolaborasi bertahap, transparan, mudah dipahami dan di-deb

Tampilkan kode

Your IPYNB file is ready

BERT_Kelompok12_A...

IPYNB

Buka

Seluruh isi arsip `Bert-Kelompok` n dipelajari. Berdasarkan kontrak teknis di `SPEC.md` v1.0 dan imple positori (`data/` , `model/` , dan

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

`.ipynb`) untuk **Bima (Anggota 3 — Training Loop & Hyperparameter Tuning Engineer)**.

Bert-Kelompok12.zip

Ringkasan Hasil Pemeriksaan Repositori Proyek

1. Spesifikasi & Konfigurasi (`SPEC.md` & `config.yaml`): Bert-Kelompok12.zip

   Task & Masking: Masked Language Modeling (MLM) murni dengan probabilitas *masking* `15%` (aturan **80%** `[MASK]` , **10%** *random token* pada rentang `[5, len(tokenizer))` , **10%** tetap) dan `ignore_index = -100` .

   Vocabulary & Special Tokens: Word-level tokenizer (`vocab_size = 30.000` , `max_seq_length = 128`) dengan ID `0–4` terkunci (`[PAD]=0, [UNK]=1, [CLS]=2, [SEP]=3, [MASK]=4`).

   Arsitektur `BERT` **(** `model/model.py` **):** `hidden_size=256` , `num_layers=6` , `num_heads=8` (`head_dim=32`), `intermediate_size=1024` , `dropout=0.1` , aktivasi `gelu` , `tie_weights=False` , menghasilkan tepat **20.233.520 parameter (~20.23M)**.

2. Modul Pelatihan & Evaluasi (`training/utils.py` , `training/metrics.py` , `training/train.py`): Bert-Kelompok12.zip

   `build_optimizer` memisahkan parameter *decay* (`0.01`) dan *no-decay* (`0.0` untuk `bias` dan `LayerNorm`).

   `build_scheduler` mengimplementasikan *Linear Warmup* + *Linear Decay* menggunakan `LambdaLR` .

   `EpochMetricTracker` menghitung *Cross-Entropy Loss*, *Top-1 Masked Token Accuracy*, dan *Perplexity* ($\exp(\text{loss})$) secara tertimbang terhadap jumlah token yang di-mask per batch.

   `training/train.py` mendukung argumen CLI override (`--run-name` , `--lr` , `--batch-size` , `--warmup-ratio` , `--weight-decay` , `--epochs` , `--eval-test` , `--max-steps`) tanpa mengubah `config.yaml` asli, serta menyimpan metrik ke `outputs/logs/<run_name>.json` & `.csv` dan kurva ke `outputs/figures/` .

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

Notebook terdiri dari **17 sel (Markdown & Code)** yang dirancang berurutan tanpa ketergantungan GitHub (`git clone` / `git push` ditiadakan sepenuhnya):

Bagian 1 — Header & Konteks Tugas Anggota 3 (Markdown): Memuat identitas proyek, peran Anggota 3, dan tabel kontrak spesifikasi `SPEC.md` v1.0.

Bagian 2 — Setup Environment & Direktori Kerja Proyek (Markdown + Code): Mengecek ketersediaan GPU (`torch.cuda.is_available()`), mengekstrak `Bert-Kelompok12.zip` secara otomatis bila dijalankan di Google Colab, mengatur *working directory* ke folder proyek, dan memverifikasi dependensi `requirements.txt` .

Bagian 3 — Inspeksi Konfigurasi, Data Pipeline & Sanity Check 1 Batch (Markdown + 2 Code Cells): Memuat `config.yaml` , memuat dataset WikiText-2 (`train` , `validation` , `test`) dan `WordTokenizer` , menampilkan tabel ukuran *split* & jumlah batch, menginisialisasi `BERT.from_config(cfg)` , serta menjalankan *forward pass* 1 batch tanpa menyimpan checkpoint untuk memvalidasi shape `logits (B, 128, 30000)` dan *initial loss* $\approx \ln(30000) \approx 10.31$.

Bagian 4 — Pelatihan Utama / Baseline Training (Markdown + 2 Code Cells): Menjalankan `training/train.py` dengan konfigurasi baseline (`lr=5e-4` , `batch_size=32` , `weight_decay=0.01` , `warmup_ratio=0.10` , `epochs=10` , `--run-name bert_mlm_baseline --eval-test`) dengan *real-time streaming log* dan menampilkan kurva `outputs/figures/training_curves.png` .

Bagian 5 — Eksperimen Hyperparameter Tuning (Markdown + Code): Menjalankan 3 skenario eksperimen (`tune_lr_1e4` , `tune_lr_1e3` , dan `tune_bs16_warmup05`) melalu *override* konfigurasi, menyimpan log JSON/CSV masing-masing ke `outputs/logs/` , dan otomatis membersihkan file checkpoint `.pt` sementara dari run tuning agar kapasitas disk Colab tetap lega.

Bagian 6 — Tabel Komparasi Hasil Tuning & Visualisasi Perbandingan Antar-Run (Markdown + 2 Code Cells): Membaca seluruh log JSON di `outputs/logs/` , menyusun `DataFrame` komparasi metrik lengkap, mengekspornya ke `outputs/logs/hyperparameter_tuning_summary.csv` , serta mem-plot grafik gabungan 3 panel ke `outputs/figures/hyperparameter_tuning_comparison.png` .

Bagian 7 — Uji Inferensi Kualitatif Prediksi Token `[MASK]` & Kesimpulan (Markdown + Code + Markdown): Memuat bobot terbaik `checkpoints/best_model.pt` dan

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

Detail Isi Sel Notebook

Berikut adalah isi lengkap dari setiap sel di dalam file `.ipynb` yang telah dibuat:

Sel 1 [Markdown] — Header & Konteks Tugas Anggota 3

Markdown

```
# Pelatihan Model, Evaluasi Metrik MLM & Eksperimen Hyperparameter Tuning – BERT-fr

**Identitas & Peran:**
- **Nama / Peran:** Bima (Anggota 3) – *Training Loop & Hyperparameter Tuning Engineer*
- **Metode Pengembangan:** *Vibe Coding* (kolaborasi bertahap, modular, transparan,
- **Target Hardware:** Google Colab (GPU NVIDIA T4 16GB) atau Jupyter Notebook Loka

---

### Ringkasan Kontrak Spesifikasi (`SPEC.md` v1.0 & `config.yaml`)

Notebook ini mengintegrasikan dan mendemonstrasikan modul-modul yang telah dibangun

| Komponen / Modul | Spesifikasi Kunci (`SPEC.md` v1.0) | File Modul |
| :--- | :--- | :--- |
| **Task Utama** | Masked Language Modeling (MLM) Murni (aturan masking 15%: **80%*
| **Dataset & Tokenizer** | **WikiText-2** (`wikitext-2-raw-v1`), `WordTokenizer` k
| **Arsitektur BERT** | `hidden_size=256`, `num_layers=6`, `num_heads=8` (`head_dim
| **Fungsi Loss & Metrik** | `CrossEntropyLoss(ignore_index=-100)`, **Masked LM Top
| **Konfigurasi Training Baseline** | `AdamW` (`lr=5e-4`, `weight_decay=0.01` denga
```

Sel 2 & 3 [Markdown + Code] — Setup Environment & Ekstraksi Folder Proyek

Python

```
import os
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```python
import torch

# 1. Cek ketersediaan GPU & informasi perangkat
print("=" * 65)
print("PEMERIKSAAN HARDWARE & PYTORCH ENVIRONMENT")
print("=" * 65)
print(f"PyTorch Version       : {torch.__version__}")
print(f"CUDA Available        : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU Device Name       : {torch.cuda.get_device_name(0)}")
    vram_gb = torch.cuda.get_device_properties(0).total_memory / (1024 ** 3)
    print(f"Total VRAM            : {vram_gb:.2f} GB")
else:
    print("PERINGATAN: GPU tidak terdeteksi. Pelatihan akan berjalan di CPU.")

# 2. Opsi ekstraksi otomatis file .zip jika dijalankan di Google Colab / folder lua
ZIP_CANDIDATES = ["Bert-Kelompok12.zip", "/content/Bert-Kelompok12.zip"]
if not Path("config.yaml").exists() and not Path("Bert-Kelompok12/config.yaml").exi
    for zpath in ZIP_CANDIDATES:
        if Path(zpath).exists():
            print(f"\nMengekstrak arsip proyek dari: {zpath} ...")
            with zipfile.ZipFile(zpath, "r") as zf:
                zf.extractall(".")
            print("Ekstraksi selesai.")
            break

# 3. Atur working directory ke root proyek (tempat config.yaml berada)
if Path("Bert-Kelompok12/config.yaml").exists():
    os.chdir("Bert-Kelompok12")

PROJECT_ROOT = Path.cwd().resolve()
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print(f"\nWorking Directory     : {PROJECT_ROOT}")
assert (PROJECT_ROOT / "config.yaml").exists(), (
    "File config.yaml tidak ditemukan! Pastikan file Bert-Kelompok12.zip sudah diur
    "atau notebook dijalankan di dalam folder root proyek."
)

# 4. Pastikan paket dependensi utama tersedia
try:
    import yaml, datasets, tqdm, matplotlib, pandas
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    os.system(f"{sys.executable} -m pip install -q -r requirements.txt pandas")
    print("Instalasi selesai.")
print("=" * 65)
```

Sel 4, 5 & 6 [Markdown + 2 Code Cells] — Inspeksi Konfigurasi, Data Pipeline & Sanity Check 1 Batch

Python

```
import math
from IPython.display import display
import pandas as pd

from data.tokenizer import load_config, build_tokenizer, WordTokenizer, report_oov
from data.data import load_dataset, MLMDataset, make_dataloader
from model.model import BERT
from training.metrics import compute_mlm_accuracy, compute_perplexity
from training.utils import set_seed, get_device

# 1. Muat konfigurasi & kunci seed
cfg = load_config("config.yaml")
set_seed(cfg["training"]["seed"])
device = get_device()

print("[1] Konfigurasi Training Baseline (config.yaml):")
for k, v in cfg["training"].items():
    print(f"    - {k:<18}: {v}")

# 2. Muat dataset WikiText-2 & WordTokenizer
print("\n[2] Memuat Dataset WikiText-2 & WordTokenizer...")
lines = load_dataset(cfg)
vocab_path = Path(cfg["paths"]["vocab_path"])
tokenizer = WordTokenizer.load(cfg) if vocab_path.exists() else build_tokenizer(cfg

print(f"    - Ukuran vocab aktif / config : {len(tokenizer):,} / {cfg['data']['voca
print(f"    - Special Token IDs           : {tokenizer.special_to_id}")
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    for split in ("train", "validation", "test")
}
dataloaders_map = {
    split: make_dataloader(datasets_map[split], cfg, shuffle=(split == "train"))
    for split in ("train", "validation", "test")
}

split_summary = pd.DataFrame([
    {
        "Split": split,
        "Baris Paragraf Bersih": len(lines[split]),
        "Sampel Chunk (L=128)": len(datasets_map[split]),
        "Jumlah Batch (bs=32)": len(dataloaders_map[split]),
    }
    for split in ("train", "validation", "test")
])
display(split_summary)
```

Python

```
# 4. Inisialisasi Model BERT & Sanity Check 1 Batch (Tanpa Menyimpan Checkpoint)
model_sanity = BERT.from_config(cfg).to(device)
total_params = model_sanity.count_parameters(trainable_only=False)
trainable_params = model_sanity.count_parameters(trainable_only=True)

print("=" * 65)
print("VERIFIKASI ARSITEKTUR BERT & SANITY CHECK 1 BATCH")
print("=" * 65)
print(f"Total Parameter       : {total_params:,} (~{total_params / 1e6:.2f}M)")
print(f"Trainable Parameter   : {trainable_params:,}")

# Ambil 1 batch dari train_loader
sample_batch = next(iter(dataloaders_map["train"]))
input_ids = sample_batch["input_ids"].to(device)
attention_mask = sample_batch["attention_mask"].to(device)
token_type_ids = sample_batch["token_type_ids"].to(device)
labels = sample_batch["labels"].to(device)
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
        input_ids=input_ids,
        attention_mask=attention_mask,
        token_type_ids=token_type_ids,
        labels=labels,
    )

expected_loss = math.log(cfg["model"]["vocab_size"])
init_loss = out.loss.item()
num_correct, num_masked, init_acc = compute_mlm_accuracy(
    out.logits, labels, ignore_index=cfg["training"]["ignore_index"]
)

print(f"\nShape input_ids       : {tuple(input_ids.shape)} | dtype: {input_ids.dtyp
print(f"Shape attention_mask  : {tuple(attention_mask.shape)} | dtype: {attention_m
print(f"Shape labels          : {tuple(labels.shape)} | dtype: {labels.dtype}")
print(f"Shape logits          : {tuple(out.logits.shape)} | dtype: {out.logits.dtyp
print("-" * 65)
print(f"Token ter-mask (batch): {num_masked} dari {input_ids.numel()} token ({num_m
print(f"Initial Loss (1 batch): {init_loss:.4f} (Teoretis ln(30000) ≈ {expected_los
print(f"Initial Top-1 Accuracy: {init_acc * 100:.4f}% ({num_correct}/{num_masked} b
print(f"Initial Perplexity    : {compute_perplexity(init_loss):.2f}")
print("=" * 65)

# Bersihkan model sanity check dari memori GPU sebelum pelatihan utama
del model_sanity, out
if torch.cuda.is_available():
    torch.cuda.empty_cache()
```

Sel 7, 8 & 9 [Markdown + 2 Code Cells] — Pelatihan Utama (Baseline Training Sesuai `SPEC.md`)

Python

```
import subprocess
from IPython.display import Image, display

# Set ke None untuk pelatihan penuh (Full Training 10 Epoch sesuai SPEC.md).
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```python
cmd_baseline = [
    sys.executable, "training/train.py",
    "--config", "config.yaml",
    "--run-name", "bert_mlm_baseline",
    "--lr", "5e-4",
    "--batch-size", "32",
    "--weight-decay", "0.01",
    "--warmup-ratio", "0.10",
    "--epochs", str(BASELINE_EPOCHS),
    "--eval-test",
]
if SMOKE_TEST_MAX_STEPS is not None:
    cmd_baseline += ["--max-steps", str(SMOKE_TEST_MAX_STEPS)]

print("Menjalankan perintah:", " ".join(cmd_baseline))
print("-" * 72)

# Streaming output langsung ke sel notebook agar progress per epoch terlihat real-t
with subprocess.Popen(
    cmd_baseline,
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    text=True,
    bufsize=1,
) as proc:
    for line in proc.stdout:
        print(line, end="")

if proc.returncode != 0:
    raise RuntimeError(f"Pelatihan baseline gagal dengan kode keluar {proc.returnco
```

Python

```python
# Tampilkan grafik kurva pelatihan baseline (outputs/figures/training_curves.png)
baseline_fig_path = Path(cfg["paths"]["figures_dir"]) / "training_curves.png"
if baseline_fig_path.exists():
    print(f"Menampilkan grafik kurva pelatihan baseline: {baseline_fig_path}")
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
else:
    print(f"File grafik {baseline_fig_path} belum ditemukan.")
```

Sel 10 & 11 [Markdown + Code] — Eksperimen Hyperparameter Tuning (3 Skenario)

Python

```
# Daftar 3 skenario eksperimen Hyperparameter Tuning
# Catatan: Anda dapat mengatur TUNING_EPOCHS (default mengikuti BASELINE_EPOCHS aga
TUNING_EPOCHS = BASELINE_EPOCHS

tuning_scenarios = [
    {
        "run_name": "tune_lr_1e4",
        "description": "LR Konservatif (1e-4), BS=32, Warmup=10%",
        "lr": 1e-4,
        "batch_size": 32,
        "warmup_ratio": 0.10,
        "weight_decay": 0.01,
    },
    {
        "run_name": "tune_lr_1e3",
        "description": "LR Agresif (1e-3), BS=32, Warmup=10%",
        "lr": 1e-3,
        "batch_size": 32,
        "warmup_ratio": 0.10,
        "weight_decay": 0.01,
    },
    {
        "run_name": "tune_bs16_warmup05",
        "description": "Batch Size Kecil (16) & Warmup Singkat (5%), LR=5e-4",
        "lr": 5e-4,
        "batch_size": 16,
        "warmup_ratio": 0.05,
        "weight_decay": 0.01,
    },
]
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
for idx, sc in enumerate(tuning_scenarios, start=1):
    print("\n" + "#" * 72)
    print(f"[EKSPERIMEN TUNING {idx}/{len(tuning_scenarios)}] {sc['run_name']} - {s
    print("#" * 72)

    cmd_tune = [
        sys.executable, "training/train.py",
        "--config", "config.yaml",
        "--run-name", sc["run_name"],
        "--lr", str(sc["lr"]),
        "--batch-size", str(sc["batch_size"]),
        "--warmup-ratio", str(sc["warmup_ratio"]),
        "--weight-decay", str(sc["weight_decay"]),
        "--epochs", str(TUNING_EPOCHS),
        "--eval-test",
    ]
    if SMOKE_TEST_MAX_STEPS is not None:
        cmd_tune += ["--max-steps", str(SMOKE_TEST_MAX_STEPS)]

    with subprocess.Popen(
        cmd_tune,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True,
        bufsize=1,
    ) as proc:
        for line in proc.stdout:
            print(line, end="")

    if proc.returncode != 0:
        raise RuntimeError(f"Eksperimen {sc['run_name']} gagal dengan kode keluar {

    # Hapus checkpoint sementara khusus run tuning agar disk Colab tidak penuh
    # (Log JSON/CSV dan kurva PNG tetap tersimpan di outputs/)
    for temp_ckpt in [
        ckpt_dir / f"best_model_{sc['run_name']}.pt",
        ckpt_dir / f"last_model_{sc['run_name']}.pt",
    ]:
        if temp_ckpt.exists():
            temp_ckpt.unlink()
            print(f"[Disk Cleanup] Menghapus checkpoint sementara: {temp_ckpt}")
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

Sel 12, 13 & 14 [Markdown + 2 Code Cells] — Tabel Komparasi & Visualisasi Perbandingan Antar-Run

Python

```python
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Image, display

logs_dir = Path(cfg["paths"]["logs_dir"])
figures_dir = Path(cfg["paths"]["figures_dir"])
figures_dir.mkdir(parents=True, exist_ok=True)

run_order = [
    "bert_mlm_baseline",
    "tune_lr_1e4",
    "tune_lr_1e3",
    "tune_bs16_warmup05",
]

loaded_runs = {}
summary_rows = []

for rname in run_order:
    jpath = logs_dir / f"{rname}.json"
    if not jpath.exists():
        print(f"Catatan: File log {jpath} belum tersedia, dilewati.")
        continue

    with open(jpath, "r", encoding="utf-8") as f:
        payload = json.load(f)

    history = payload.get("history", [])
    if not history:
        continue

    loaded_runs[rname] = payload
    hp = payload.get("hyperparameters", {})
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    test_metrics = payload.get("final_test_metrics", {})

    summary_rows.append({
        "Run Name": rname,
        "LR": hp.get("learning_rate", "-"),
        "Batch Size": hp.get("batch_size", "-"),
        "Warmup Ratio": hp.get("warmup_ratio", "-"),
        "Best Epoch": best_row["epoch"],
        "Train Loss (@Best)": round(best_row["train_loss"], 4),
        "Best Val Loss": round(best_row["val_loss"], 4),
        "Best Val Acc (%)": round(best_row["val_accuracy"] * 100.0, 2),
        "Best Val PPL": round(best_row["val_perplexity"], 2),
        "Test Acc (%)": (
            round(test_metrics["accuracy"] * 100.0, 2)
            if "accuracy" in test_metrics else "-"
        ),
        "Test PPL": (
            round(test_metrics["perplexity"], 2)
            if "perplexity" in test_metrics else "-"
        ),
        "Avg Epoch Time (s)": round(avg_time, 2),
    })

comparison_df = pd.DataFrame(summary_rows)
print("=" * 80)
print("TABEL KOMPARASI HASIL EKSPERIMEN HYPERPARAMETER TUNING")
print("=" * 80)
display(comparison_df)

# Simpan juga tabel ringkasan ke CSV untuk lampiran laporan Kelompok 12
comparison_csv_path = logs_dir / "hyperparameter_tuning_summary.csv"
comparison_df.to_csv(comparison_csv_path, index=False)
print(f"Tabel komparasi disimpan ke: {comparison_csv_path}")
```

Python

```
# Plot grafik komparasi antar-run (Val Loss, Val Accuracy, Val Perplexity)
if loaded_runs:
    fig, axes = plt.subplots(1, 3, figsize=(18, 5.2))
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
        hist = payload["history"]
        hp = payload.get("hyperparameters", {})
        label = f"{rname} (lr={hp.get('learning_rate')}, bs={hp.get('batch_size')})"
        mk = markers[idx % len(markers)]

        epochs = [r["epoch"] for r in hist]
        val_loss = [r["val_loss"] for r in hist]
        val_acc = [r["val_accuracy"] * 100.0 for r in hist]
        val_ppl = [r["val_perplexity"] for r in hist]

        axes[0].plot(epochs, val_loss, marker=mk, linewidth=2, label=label)
        axes[1].plot(epochs, val_acc, marker=mk, linewidth=2, label=label)
        axes[2].plot(epochs, val_ppl, marker=mk, linewidth=2, label=label)

    axes[0].set_title("Comparison: Validation MLM Loss")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Cross-Entropy Loss")
    axes[0].grid(True, linestyle="--", alpha=0.5)
    axes[0].legend(fontsize=8.5)

    axes[1].set_title("Comparison: Validation MLM Top-1 Accuracy (%)")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Accuracy (%)")
    axes[1].grid(True, linestyle="--", alpha=0.5)
    axes[1].legend(fontsize=8.5)

    axes[2].set_title("Comparison: Validation Perplexity")
    axes[2].set_xlabel("Epoch")
    axes[2].set_ylabel("Perplexity (exp(Loss))")
    axes[2].grid(True, linestyle="--", alpha=0.5)
    axes[2].legend(fontsize=8.5)

    fig.suptitle(
        "BERT-from-Scratch (Kelompok 12) - Hyperparameter Tuning Comparison",
        fontsize=14,
        fontweight="bold",
    )
    fig.tight_layout()

    comp_fig_path = figures_dir / "hyperparameter_tuning_comparison.png"
    fig.savefig(comp_fig_path, dpi=150)
    plt.close(fig)
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

Sel 15, 16 & 17 [Markdown + Code + Markdown] — Uji Inferensi Kualitatif Prediksi Token `[MASK]` & Kesimpulan

Python

```python
import torch.nn.functional as F
from training.utils import load_checkpoint

best_ckpt_path = Path(cfg["paths"]["checkpoint_dir"]) / "best_model.pt"
assert best_ckpt_path.exists(), (
    f"Checkpoint {best_ckpt_path} belum ditemukan. Jalankan sel Pelatihan Baseline
)

# Muat model terbaik dari checkpoint
infer_model = BERT.from_config(cfg).to(device)
ckpt_info = load_checkpoint(best_ckpt_path, model=infer_model, device=device)
infer_model.eval()

print("=" * 72)
print(
    f"MODEL TERBAIK DIMUAT DARI : {best_ckpt_path} "
    f"(Epoch {ckpt_info.get('epoch')}, Best Val Loss: {ckpt_info.get('best_val_loss
)
print("=" * 72)


@torch.no_grad()
def predict_masked_sentence(
    sentence: str,
    model: BERT,
    tokenizer: WordTokenizer,
    device: torch.device,
    top_k: int = 5,
) -> pd.DataFrame:
    """
    Memprediksi Top-K kandidat token pada posisi [MASK] di dalam kalimat uji.
    Catatan: kata biasa di-lowercase agar sesuai dengan preprocessing WikiText-2,
    sedangkan token khusus [MASK] dipertahankan.
    """
    tokens = [
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    input_ids_list = tokenizer.encode(
        normalized_text,
        add_special_tokens=True,
        max_length=tokenizer.max_seq_length,
        padding=True,
    )
    input_ids = torch.tensor([input_ids_list], dtype=torch.long, device=device)
    attention_mask = (input_ids != tokenizer.pad_token_id).long()

    mask_positions = (input_ids[0] == tokenizer.mask_token_id).nonzero(as_tuple=Tru
    if not mask_positions:
        raise ValueError("Kalimat uji harus mengandung minimal satu token '[MASK]'.

    out = model(input_ids=input_ids, attention_mask=attention_mask)
    logits = out.logits[0]  # (L, V)

    records = []
    for pos in mask_positions:
        probs = F.softmax(logits[pos], dim=-1)
        top_probs, top_ids = torch.topk(probs, k=top_k)

        for rank, (tid, pval) in enumerate(zip(top_ids.tolist(), top_probs.tolist()
            pred_word = tokenizer.id_to_token.get(tid, "[UNK]")
            filled_tokens = [
                f"[{pred_word}]" if idx == (pos - 1) else tok
                for idx, tok in enumerate(tokens)
            ]
            records.append({
                "Kalimat Input": sentence,
                "Posisi [MASK]": pos,
                "Rank": rank,
                "Prediksi Token": pred_word,
                "Token ID": tid,
                "Probabilitas (%)": round(pval * 100.0, 2),
                "Kalimat Hasil Prediksi": " ".join(filled_tokens),
            })

    return pd.DataFrame(records)


test_sentences = [
    "the album was released in [MASK] by the record label .",
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    "she played the [MASK] and sang lead vocals on the track .",
]

for idx, sent in enumerate(test_sentences, start=1):
    print(f"\n[{idx}] Input : {sent}")
    pred_df = predict_masked_sentence(sent, infer_model, tokenizer, device=device,
    display(pred_df[["Rank", "Prediksi Token", "Token ID", "Probabilitas (%)", "Kal
```

Apakah ada skenario *hyperparameter* tambahan (seperti variasi `weight_decay` atau `dropout`) yang ingin Anda tambahkan ke dalam daftar eksperimen di notebook ini?

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini